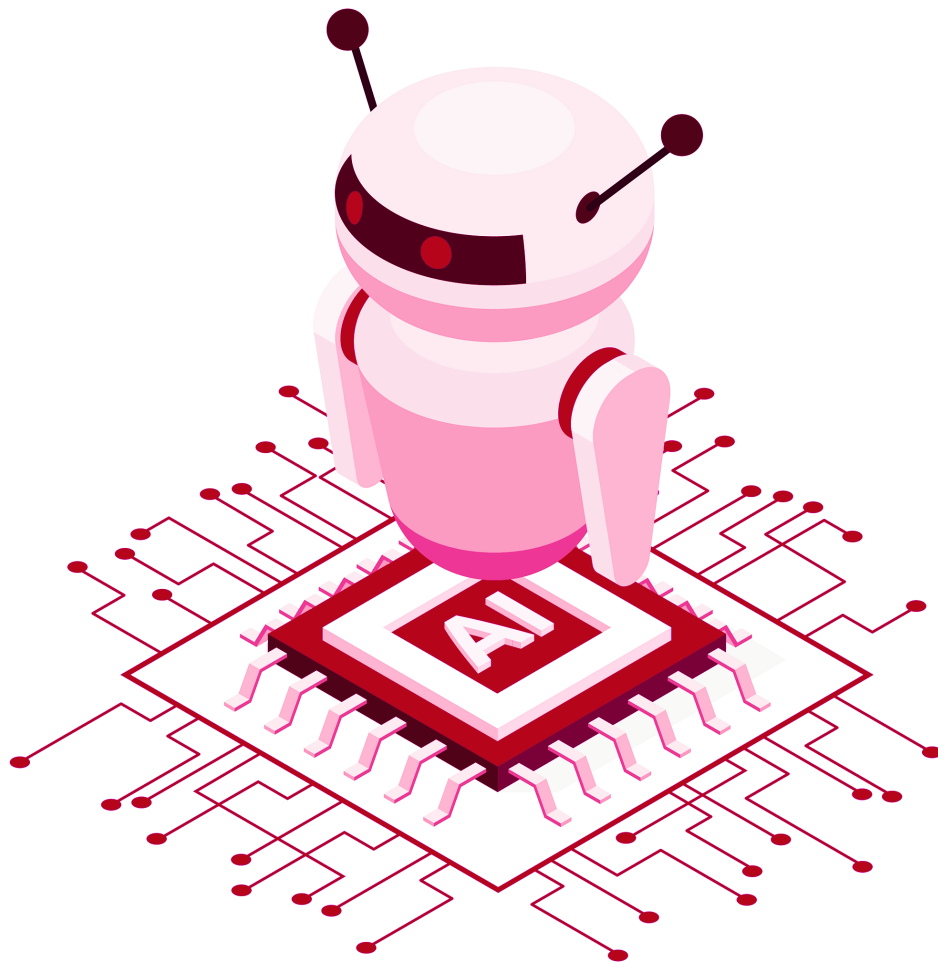




Deep Reinforcement Learning

Professor Mohammad Hossein Rohban

Solution for Homework 13:

Multi-Agent RL

By:

Parsa Ghezelbash
401110437



Spring 2025

Grading

The grading will be based on the following criteria, with a total of 110 points:

Task	Points
Task 1	50
Task 2	50
Clarity and Quality of Code	5
Clarity and Quality of Report	5
Bonus 1	5
Bonus 2	5

Contents

1	Part 1: Game Theory Problems	1
1.1	Standard Rock-Scissors-Paper	1
1.2	Modified Rock-Scissors-Paper	2
2	Part 2: Implementing MADDPG/IDDPG	3

1 Part 1: Game Theory Problems

1.1 Standard Rock-Scissors-Paper

Let Player 1's mixed strategy be $\sigma_1 = (p_R, p_S, p_P)$ and Player 2's be $\sigma_2 = (q_R, q_S, q_P)$. Since probabilities must sum to 1, we can express one in terms of the others: $p_P = 1 - p_R - p_S$ and $q_P = 1 - q_R - q_S$.

Let $U_1(\sigma_1, \sigma_2)$ be the expected utility for Player 1, and $U_2(\sigma_1, \sigma_2)$ for Player 2. These are calculated as $\sigma_1 \cdot A \cdot \sigma_2^T$ where A is Player 1's payoff matrix.

$$U_1(\sigma_1, \sigma_2) = \begin{pmatrix} p_R & p_S & p_P \end{pmatrix} \begin{pmatrix} 0 & 1 & -1 \\ -1 & 0 & 1 \\ 1 & -1 & 0 \end{pmatrix} \begin{pmatrix} q_R \\ q_S \\ q_P \end{pmatrix}$$

Expanding this gives:

$$U_1(\sigma_1, \sigma_2) = p_R(q_S - q_P) + p_S(q_P - q_R) + p_P(q_R - q_S)$$

Substitute $p_P = 1 - p_R - p_S$ and $q_P = 1 - q_R - q_S$:

$$U_1 = p_R(q_S - (1 - q_R - q_S)) + p_S((1 - q_R - q_S) - q_R) + (1 - p_R - p_S)(q_R - q_S)$$

$$U_1(p_R, p_S, q_R, q_S) = p_R(q_R + 2q_S - 1) + p_S(1 - 2q_R - q_S) + (1 - p_R - p_S)(q_R - q_S)$$

After expanding and collecting terms:

$$U_1(p_R, p_S, q_R, q_S) = p_R(3q_S - 1) + p_S(1 - 3q_R) + q_R - q_S$$

$$\frac{\partial U_1}{\partial p_R} = 3q_S - 1$$

$$\frac{\partial U_1}{\partial p_S} = 1 - 3q_R$$

Set these to zero to solve for q_R and q_S :

$$3q_S - 1 = 0 \implies q_S = \frac{1}{3}$$

$$1 - 3q_R = 0 \implies q_R = \frac{1}{3}$$

Now find q_P :

$$q_P = 1 - q_R - q_S = 1 - \frac{1}{3} - \frac{1}{3} = \frac{1}{3}$$

This gives Player 2's equilibrium strategy: $\sigma_2^* = (\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$.

The game is zero-sum, so $U_2(\sigma_1, \sigma_2) = -U_1(\sigma_1, \sigma_2)$.

$$U_2(p_R, p_S, q_R, q_S) = -[p_R(3q_S - 1) + p_S(1 - 3q_R) + q_R - q_S]$$

This gives Player 1's equilibrium strategy: $\sigma_1^* = (\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$.

1.2 Modified Rock-Scissors-Paper

Let Player 1's mixed strategy be $\sigma_1 = (p_R, p_S, p_P)$ and Player 2's be $\sigma_2 = (q_R, q_S, q_P)$. We use the constraints $p_P = 1 - p_R - p_S$ and $q_P = 1 - q_R - q_S$.

The new payoff matrix for Player 1 (A) is:

$$A = \begin{pmatrix} 0 & 1 & -2 \\ -1 & 0 & 3 \\ 2 & -3 & 0 \end{pmatrix}$$

The expected utility for Player 1, $U_1(\sigma_1, \sigma_2) = \sigma_1 \cdot A \cdot \sigma_2^T$, is:

$$U_1 = p_R(0 \cdot q_R + 1 \cdot q_S - 2 \cdot q_P) + p_S(-1 \cdot q_R + 0 \cdot q_S + 3 \cdot q_P) + p_P(2 \cdot q_R - 3 \cdot q_S + 0 \cdot q_P)$$

$$U_1 = p_R(q_S - 2q_P) + p_S(-q_R + 3q_P) + p_P(2q_R - 3q_S)$$

Substitute $p_P = 1 - p_R - p_S$ and $q_P = 1 - q_R - q_S$:

$$U_1 = p_R(q_S - 2(1 - q_R - q_S)) + p_S(-q_R + 3(1 - q_R - q_S)) + (1 - p_R - p_S)(2q_R - 3q_S)$$

Expand the terms:

$$U_1 = p_R(q_S - 2 + 2q_R + 2q_S) + p_S(-q_R + 3 - 3q_R - 3q_S) + (2q_R - 3q_S - 2p_Rq_R + 3p_Rq_S - 2p_Sq_R + 3p_Sq_S)$$

$$U_1 = p_R(2q_R + 3q_S - 2) + p_S(-4q_R - 3q_S + 3) + 2q_R - 3q_S - 2p_Rq_R + 3p_Rq_S - 2p_Sq_R + 3p_Sq_S$$

Collect terms for p_R and p_S :

$$U_1(p_R, p_S, q_R, q_S) = p_R(4q_R + 6q_S - 2) + p_S(-6q_R - 6q_S + 3) + 2q_R - 3q_S$$

To find Player 2's equilibrium strategy, we take the partial derivatives of U_1 with respect to Player 1's strategy variables (p_R, p_S) and set them to zero. This ensures Player 1 is indifferent between their choices.

$$\frac{\partial U_1}{\partial p_R} = 6q_S - 2$$

$$\frac{\partial U_1}{\partial p_S} = -6q_R + 3$$

Setting these to zero gives a system of linear equations:

$$\begin{cases} 6q_S = 2 & (1) \\ 6q_R = 3 & (2) \end{cases}$$

Player 2's equilibrium strategy is $\sigma_2^* = (\frac{1}{2}, \frac{1}{3}, \frac{1}{6})$.

The game is zero-sum, so Player 2's utility is $U_2 = -U_1$.

$$U_2(p_R, p_S, q_R, q_S) = -[p_R(4q_R + 6q_S - 2) + p_S(-6q_R - 6q_S + 3) + 2q_R - 3q_S]$$

Player 1's equilibrium strategy is $\sigma_1^* = (\frac{1}{2}, \frac{1}{3}, \frac{1}{6})$.

2 Part 2: Implementing MADDPG/IDDPG

1. In our training loop, the DDPGLoss module utilizes `target_policies` to estimate the value of the next state. Explain clearly why employing these slowly-updating target networks, rather than the main policy networks (which change rapidly), is essential for ensuring the stability of the DDPG algorithm. (Hint: Consider what might happen if the critic tried to optimize toward a continuously moving target.)

Answer:

In Deep Deterministic Policy Gradient (DDPG), the critic network is trained by minimizing the loss between its current Q-value prediction, $Q(s, a; \theta_Q)$, and a target value, y_{target} . If we were to use the main networks for this target, the update rule would be:

$$y_{\text{target}} = r + \gamma Q(s', \mu(s'; \theta_\mu); \theta_Q)$$

The parameters of the critic, θ_Q , are updated based on a target that is also a function of θ_Q . This creates a non-stationary or "moving" target. As θ_Q is updated via gradient descent, the target value shifts, leading to optimization instability, oscillations, and potential divergence.

To solve this, we introduce separate **target networks**, $Q'(s, a; \theta_{Q'})$ and $\mu'(s'; \theta_{\mu'})$, whose weights are a slowly-updating copy of the main network weights. The target value is then calculated as:

$$y_{\text{target}} = r + \gamma Q'(s', \mu'(s'; \theta_{\mu'}); \theta_{Q'})$$

The target network weights are updated via Polyak averaging:

$$\theta' \leftarrow \tau \theta + (1 - \tau) \theta' \quad \text{for a small } \tau$$

This method stabilizes training by providing a temporarily fixed and consistent target for the critic's loss function.

2. (bonus) Consider the training plot shown in Figure 1, which resulted from modifying a single scalar hyper-parameter in the training script.
 - (a) Describe the issue with the learning process depicted in the plot.
 - (b) Identify which hyper-parameter you believe was changed, and explain the role of this parameter within the MADDPG algorithm.

Answer:

The learning process depicted in the plot is characterized by an initial performance collapse followed by extremely slow and inefficient learning. The reward first drops to a low of -7, indicating that the initial policy updates are not ideal. Subsequently, the reward recovers and climbs very gradually to a suboptimal value of 2.5 over the course of training. This behavior suggests that the learning updates are too small or infrequent, preventing the agent from making meaningful progress and exploring the policy space effectively.

Given the slow learning rate, the single scalar hyper-parameter most likely modified is the **actor's learning rate (LR)**, but in this case, it was set to be **too low**.

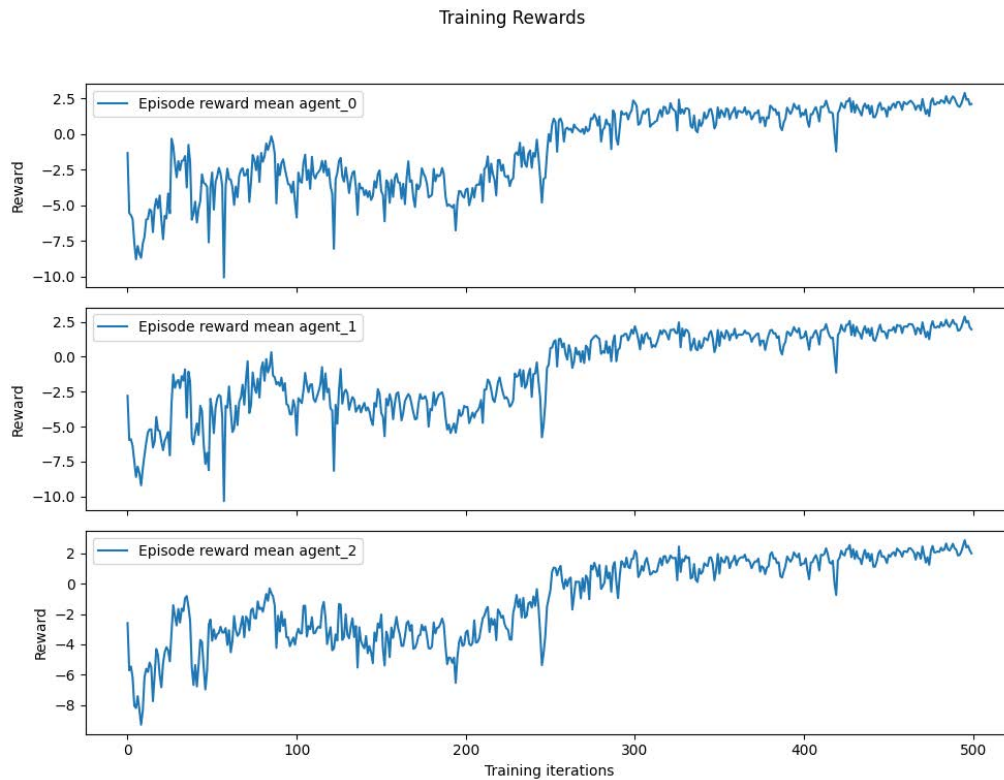


Figure 1: Agents performance after modifying a scalar hyper-parameter.

References

- [1] [Cover image designed by freepik](#)
- [2] Ryan Lowe, Yi I. Wu, Aviv Tamar, Jean Harb, Pieter Abbeel, and Igor Mordatch. Multi-agent actor-critic for mixed cooperative-competitive environments. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. [arXiv:1706.02275](#)